Product Requirements Document (PRD)

Project: AI-Powered Network Intrusion Detection System (JSON API Edition)

1 Executive Summary & System Overview

Modern enterprise networks generate massive volumes of packet traffic, exposing infrastructure to distributed denial-of-service (DDoS), brute-force credential stuffing, and botnet communication. Traditional signature-based intrusion detection systems (IDS) rely on static, human-crafted rules that fail to generalize against evolving threat vectors.

This project specifies the architecture and multi-stage implementation of an **AI-Powered Network Intrusion Detection System (NIDS)**. The system ingests standardized bidirectional network flow records via a JSON REST API, cleans and validates the payload, and trains machine learning models to perform binary classification: identifying traffic flows as **Normal (BENIGN)** or **Malicious (ATTACK)**. Models learn the patterns of known attack types in the training data and are evaluated on how well they generalize to unseen flows.

The delivery lifecycle spans three distinct engineering phases mapped directly to the core computational learning outcomes:

- **Phase 1 (ETL & EDA):** Programmatic API extraction of JSON payloads, pandas cleaning, Pydantic schema validation, SQLite persistent storage, and a deployed visual EDA dashboard.
- **Phase 2 (Classical ML):** A unified scikit-learn `Pipeline` encapsulating preprocessing and an ensemble classifier, tuned with cross-validated search and evaluated on a strictly held-out test partition.
- **Phase 3 (Deep Learning):** A PyTorch neural network tuned via Optuna, with end-to-end parameter, metric, and serialized model artifact logging managed in MLflow.

2 Course Alignment & Timeline Matrix

#	Phase	Timeline	CLO	Marks	Deliverable
1	ETL & EDA Pipeline	Weeks 6–8	CLO2	10	REST API JSON ingestion, pandas cleaning, Pydantic validation, SQLite loading, and a structured EDA dashboard.
2	scikit-learn ML Pipeline	Weeks 9–10	CLO3	10	Single atomic `Pipeline`; cross-validated grid search; evaluation on a strictly sequestered 20% test set.
3	PyTorch & MLflow	Weeks 12–15	CLO4	10	PyTorch neural network, Optuna sweeps, complete MLflow tracking (parameters, metrics, and the final model), and evaluation.
			Total	**30**	

3 Project 1: Data Ingestion, ETL & EDA (Weeks 6–8)

3.1 Dataset

The data is the CICIDS2017 flow dataset (the "MachineLearningCVE" CSV files from the Canadian Institute for Cybersecurity), with roughly 2.8 million labeled flows and about 78 numeric features per flow. Known characteristics the pipeline must handle: column names with stray leading spaces, infinite and missing values in rate columns (`Flow Bytes/s`, `Flow Packets/s`), many duplicate rows, and strong class imbalance (the large majority of flows are benign, and some attack classes are very rare).

3.2 REST API Ingestion (JSON Format)

To satisfy the requirement of extracting data from an API, the project includes a small local FastAPI **source API** that serves the CICIDS2017 flow records as JSON in batches (`GET /flows`, with `offset` and `limit` query parameters). The ETL script uses Python's `requests` library to page through this endpoint and collect the records.

```python
import requests

def fetch_network_flows_json(api_url: str, offset: int, limit: int = 5000):
    """Fetches one batch of network flows as a JSON payload."""
    response = requests.get(
        api_url, params={"offset": offset, "limit": limit}, timeout=15
    )
    response.raise_for_status()
    # Parses the response directly into a Python list of dictionaries
    return response.json()
```

3.3 Data Cleaning (pandas)

Each fetched batch is loaded into a pandas `DataFrame` and transformed before validation:

- Strip whitespace from column names and normalize them to snake_case.
- Replace $\pm\infty$ values with `NaN`, then handle missing values (drop or impute, documented in the notebook).
- Drop exact duplicate rows.
- Downcast numeric dtypes to reduce memory use on the large table.

3.4 Data Validation (Pydantic Schema)

Cleaned records are passed into strongly-typed Pydantic models to guarantee data integrity. The model below shows the pattern; the full schema covers all numeric feature columns kept for Project 2, not only the fields shown.

```python
from pydantic import BaseModel, Field, field_validator
import math

class NetworkFlow(BaseModel):
    destination_port: int = Field(..., ge=0, le=65535)
    flow_duration: int = Field(..., ge=0)
    total_fwd_packets: int = Field(..., ge=0)
    total_backward_packets: int = Field(..., ge=0)
    packet_length_mean: float = Field(..., ge=0)
```

```
    flow_bytes_s: float
    flow_packets_s: float
    # ... remaining numeric feature columns follow the same pattern ...
    label: str

    @field_validator("flow_bytes_s", "flow_packets_s", mode="before")
    @classmethod
    def clean_inf_and_nan(cls, v):
        if v is None or math.isnan(float(v)) or math.isinf(float(v)):
            return 0.0
        return float(v)
```

3.5 Target Variable & Schema Loading (SQLite)

Multiclass labels are standardized into a binary indicator (0 for BENIGN, 1 for ATTACK). The original label is kept alongside it so the EDA can break attacks down by type. Validated records are loaded idempotently into an SQLite database.

```
CREATE TABLE network_flows (
    flow_id TEXT PRIMARY KEY,
    destination_port INTEGER NOT NULL,
    flow_duration INTEGER NOT NULL,
    total_fwd_packets INTEGER NOT NULL,
    total_backward_packets INTEGER NOT NULL,
    packet_length_mean REAL NOT NULL,
    flow_bytes_s REAL NOT NULL,
    flow_packets_s REAL NOT NULL,
    -- ... remaining numeric feature columns ...
    label TEXT NOT NULL,
    is_attack INTEGER NOT NULL
);
CREATE INDEX idx_attack_flag ON network_flows(is_attack);
```

3.6 Exploratory Data Analysis (EDA)

The EDA is presented via a lightweight FastAPI web dashboard with static HTML/JS charts. (Streamlit is not used because it needs a long-running server, which Vercel's serverless runtime cannot host.) The dashboard shows:

- **Class Imbalance Analysis:** The ratio of benign to malicious traffic, and the counts per attack type.
- **Feature Correlation Heatmap:** Which features correlate most strongly with `is_attack`.
- **Volumetric Profiling:** Scatter plots of Packet Rate vs. Flow Duration illustrating the clustering of high-rate DDoS attacks vs. benign sessions.

Class counts and correlations are computed on the **full** database so they reflect the true class balance; the scatter plot uses a stratified sample. A Jupyter notebook with the same analysis documents the cleaning decisions and findings.

3.7 Delivery & Deployment

- **Local:** the source API, ETL, and full SQLite database (`nids_full.db`) run locally and are not deployed, because the full dataset exceeds Vercel's size limits.

- **Deployed:** only the dashboard is deployed to Vercel. It reads a small read-only SQLite file (`nids_deploy.db`) exported from the full database, containing a stratified sample and pre-computed summary tables.
- **Version control:** work is committed to a new branch on GitHub, reviewed, and then merged into `main`. Vercel deploys from `main`.

4 Project 2: scikit-learn ML Pipeline (Weeks 9–10)

4.1 Stratified Train / Held-Out Test Split

Features are read from the full SQLite database. Before any scaling or training occurs, the dataset is partitioned. A `stratify` argument guarantees the 20% test set contains the correct proportion of attacks. **This test set remains completely sequestered during hyperparameter tuning.**

```
from sklearn.model_selection import train_test_split

X = df.drop(columns=["is_attack", "label", "flow_id"])
y = df["is_attack"]

# 80% Train, 20% Sequestered Test Set
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

4.2 The Unified `sklearn.pipeline.Pipeline`

Data scaling and the estimator are explicitly bound into a single `Pipeline`, so the scaler is fit only on training folds during cross-validation and no information leaks from validation or test data.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

# Explicit satisfaction of the unified Pipeline requirement
nids_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(
        class_weight="balanced", random_state=42))
])
```

4.3 Cross-Validated Search & Final Evaluation

A stratified, cross-validated `GridSearchCV` (scored on F1) is executed **only on X_train**.

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

param_grid = {
    "classifier__n_estimators": [100, 200],
    "classifier__max_depth": [10, 20, None],
}
search = GridSearchCV(
    nids_pipeline, param_grid, scoring="f1",
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    n_jobs=-1,
)
search.fit(X_train, y_train)
```

The resulting `best_estimator_` is evaluated exactly once against the held-out `X_test` partition to report the definitive baseline Precision, Recall, F1, ROC-AUC, and confusion matrix.

5 Project 3: PyTorch & MLflow Lifecycle (Weeks 12–15)

5.1 Network Architecture & Optuna Tuning

A multi-layer feed-forward neural network is built using PyTorch. The same stratified train/test split from Project 2 is reused, and a validation set is carved out of the training portion. Optuna orchestrates the hyperparameter sweep, searching for the optimal learning rate, dropout rate, and hidden layer dimensionalities over multiple training trials. **Each trial is scored on the validation set; the test set is used only once, for the final model.** The result is reported as a comparison against the Project 2 baseline.

5.2 MLflow Experiment Tracking & Artifact Logging

Every Optuna trial is logged as its own MLflow run, with parameters, per-epoch training and validation loss (so the loss curve can be plotted), and the validation score. After the sweep, the best configuration is retrained, evaluated once on the test set, and **the final trained model is serialized and logged as an MLflow model artifact.**

```python
import mlflow
import mlflow.pytorch

def objective(trial):
    lr = trial.suggest_float("learning_rate", 1e-4, 1e-2, log=True)
    dropout = trial.suggest_float("dropout", 0.0, 0.5)
    with mlflow.start_run(run_name=f"trial_{trial.number}", nested=True):
        mlflow.log_params({"learning_rate": lr, "dropout": dropout})
        for epoch in range(n_epochs):
            # ... train one epoch, compute losses on train and validation ...
            mlflow.log_metric("train_loss", train_loss, step=epoch)
            mlflow.log_metric("val_loss", val_loss, step=epoch)
        mlflow.log_metric("val_f1", val_f1)
    return val_f1

# After the Optuna study: retrain with best params, evaluate once on test
with mlflow.start_run(run_name="NIDS_final_model"):
    mlflow.log_params(study.best_params)
    mlflow.log_metric("test_precision", test_precision)
    mlflow.log_metric("test_recall", test_recall)
    mlflow.log_metric("test_f1", test_f1)
    # CRITICAL: Serializing and logging the final model artifact
    mlflow.pytorch.log_model(pytorch_model, artifact_path="final_model")
```

6 Evaluation Criteria & Success Metrics

Standard Accuracy is an invalid success metric due to class imbalance. Models will be evaluated on the held-out test set using:

- **Precision:** Minimizes False Positives to avoid blocking legitimate network traffic.
- **Recall (Detection Rate):** Minimizes False Negatives to prevent security breaches.
- **F1-Score:** The harmonic mean of Precision and Recall.

- **ROC-AUC and confusion matrix:** Threshold-independent ranking quality and a direct view of the error types.

Where useful, recall is also broken down by attack type using the stored original label.

7 Out of Scope

- Real-time packet sniffing (for example, live Wireshark or pcap capture). All work is offline batch training on historical flow records.
- Large language models or generative text AI.
- Streamlit or other dashboards requiring a persistent server.